Slide 1 de 45

Base de JavaScript para OOP

Tudo que você precisa dominar antes de começar

Antes de entrar em Orientação a Objetos, precisamos ter uma base sólida de JavaScript. Este material cobre os fundamentos que serão usados **o tempo todo** no módulo de OOP.

- Variáveis e tipos de dados
- Operadores e comparações
- Estruturas condicionais
- Estruturas de repetição
- Funções (declaração, expressão, arrow)
- Arrays e seus métodos principais
- Objetos literais
- Template literals
- Desestruturação e spread
- Tratamento de erros

Se algum desses tópicos parecer novo, **pare e pratique** antes de avançar. Cada conceito aqui será usado diretamente nas aulas de OOP.

Slide 2 de 45

Variáveis — var, let e const

As três formas de declarar variáveis e quando usar cada uma

JavaScript tem três palavras-chave para declarar variáveis. Na prática moderna, usamos quase exclusivamente `let` e `const`.

```
// ❌ var – evite! Tem escopo de função
(não de bloco) e permite redeclaração
var nome = "Ana";
var nome = "Carlos"; // sem erro – isso é perigoso!

// ✅ let – para valores que vão mudar
let idade = 25;
idade = 26; // ok, pode reatribuir
// let idade = 30; // ❌ Erro! Não pode redeclarar

// ✅ const – para valores que NÃO vão mudar
const PI = 3.14159;
// PI = 3; // ❌ Erro! Não pode reatribuir

// CUIDADO: const com objetos/arrays permite mudar o CONTEÚDO
const pessoa = { nome: "Ana" };
pessoa.nome = "Bia"; // ✅ Funciona!
Mudou o conteúdo, não a referência
// pessoa = {};       // ❌ Erro!
Não pode apontar para outro objeto
```

Palavra	Reatribuir?	Redeclarar?	Escopo	Usar quando
var	Sim	Sim	Função	Nunca (código legado)
let	Sim	Não	Bloco {}	Valor vai mudar (contadores, estados)
const	Não	Não	Bloco {}	Valor não muda (padrão — use sempre que possível)

Regra prática: comece com `const`. Só troque para `let` se precisar reatribuir. Nunca use `var`.

Slide 3 de 45

Tipos de Dados Primitivos

Os 7 tipos primitivos do JavaScript

JavaScript tem 7 tipos primitivos. Primitivos são **imutáveis** — quando você "altera" uma string, na verdade cria uma nova.

```
// string — texto
const nome = "Ana";
const frase = 'Também pode usar aspas simples';
const template = `Olá, ${nome}!`; //
template literal (veremos mais adiante)

// number — inteiros e decimais (não existe int/float separado)
const idade = 25;
const preco = 19.90;
const infinito = Infinity;
const naoNumero = NaN; // Not a Number

// boolean — verdadeiro ou falso
const ativo = true;
const deletado = false;

// undefined — variável declarada mas sem valor atribuído
let endereco;
console.log(endereco); // undefined

// null — ausência intencional de valor
const resposta = null; // "propositalmente vazio"

// bigint — números inteiros muito grandes
const enorme = 9007199254740991n;

// symbol — identificador único (raro no dia a dia)
const id = Symbol("id");
```

No módulo de OOP, os mais usados serão: `string`, `number`, `boolean`, `undefined` e `null`. Use `typeof` para verificar o tipo: `typeof 42` retorna `"number"`.

Slide 4 de 45

Tipos de Dados — Referência

Objetos, arrays e funções

Além dos primitivos, temos os **tipos de referência**. A diferença crucial: primitivos são copiados por **valor**, referências são copiadas por **referência** (apontam para o mesmo lugar na memória).

```
// PRIMITIVO — cópia por valor
let a = 10;
let b = a;   // b recebe uma CÓPIA do valor
b = 20;
console.log(a); // 10 — inalterado!

// REFERÊNCIA — cópia por referência
const original = { nome: "Ana", idade: 25 };
const copia = original; // copia aponta para o MESMO objeto!
copia.idade = 30;
console.log(original.idade); // 30 — ALTEROU o original!

// Para copiar de verdade um objeto:
const copiaReal = { ...original
}; // spread operator (cópia rasa)
copiaReal.idade = 99;
console.log(original.idade); // 30 — original intacto
```

Isso é **fundamental** para OOP: quando passamos um objeto como parâmetro para um método, estamos passando a **referência**. Qualquer modificação dentro do método afeta o objeto original.

Slide 5 de 45

Verificação — Tipos e Referência

Testando o conceito de referência

Qual é o output? `const x = { a: 1 }; const y = x; y.a = 5; console.log(x.a);`

1

✓ 5 (Resposta Correta)

undefined

Erro

Explicação: Correto! `y` aponta para o mesmo objeto que `x`.
Alterar `y.a` altera o mesmo objeto, então `x.a` também é 5.

Slide 6 de 45

Operadores de Comparação

== vs === e as armadilhas do JS

JavaScript tem dois tipos de comparação de igualdade. Na prática, **sempre use `===`** (igualdade estrita) para evitar bugs sutis.

```
// == (igualdade solta) - converte tipos antes de comparar
console.log(5 == "5");    // true  😱 (string vira number)
console.log(0 == false);  // true  😱
console.log("" == false); // true  😱
console.log(null == undefined); // true 😱

// === (igualdade estrita) - compara valor E tipo
console.log(5 === "5");   // false ✅ (number ≠ string)
console.log(0 === false); // false ✅
console.log("" === false); // false ✅
console.log(null === undefined); // false ✅

// Mesma lógica para != (solto) vs !== (estrito)
console.log(5 != "5");  // false (converte, acha igual)
console.log(5 !== "5"); // true  (tipos diferentes = diferente)
```

Regra absoluta: sempre use `===` e `!==`. No módulo de OOP, faremos muitas comparações dentro de métodos e validações — usar `==` é pedir para ter bugs invisíveis.

Slide 7 de 45

Valores Truthy e Falsy

O que o JavaScript considera verdadeiro ou falso

Em JavaScript, qualquer valor pode ser avaliado como `true` ou `false` em um contexto booleano (como um `if`). Os valores que são avaliados como `false` são chamados **falsy**.

Falsy (avaliados como false)	Truthy (avaliados como true)
false	true
0 e -0	Qualquer número diferente de 0 (ex: 1, -5, 3.14)
"" (string vazia)	Qualquer string não vazia (ex: "0", "false", " ")
null	Qualquer objeto, mesmo vazio: {} e []
undefined	Qualquer função
NaN	Infinity e -Infinity

```
// Isso é muito usado em validações de OOP:
const nome = "";

if (nome) {
  console.log("Nome válido");
} else {
  console.log("Nome vazio!"); // ← entra aqui, pois "" é falsy
}

// Atalho comum com operador !
const titulo = "";
if (!titulo) {
  console.log("Título não pode ser
vazio!"); // validação de setter
}

// CUIDADO com o 0!
const estoque = 0;
if (!estoque) {
  console.log("Sem estoque"); // entra aqui! 0 é falsy
}
// Melhor: if (estoque === 0) ou if (estoque <= 0)
```

Slide 8 de 45

Verificação — Truthy e Falsy

Testando a intuição sobre valores falsy

Qual desses valores é TRUTHY (avaliado como true) em JavaScript?

0

""

✓ "0" (Resposta Correta)

null

Explicação: Exato! A string "0" é truthy porque é uma string NÃO vazia. Apenas a string vazia "" é falsy. O conteúdo dela ser o caractere zero não importa.

Slide 9 de 45

Estruturas Condicionais

if, else if, else e operador ternário

Condicionais controlam o fluxo do programa. Em OOP, usaremos muito dentro de métodos para **validações** e **regras de negócio**.

```
// if / else if / else
const saldo = 1500;
const valorSaque = 2000;

if (valorSaque <= 0) {
  console.log("Valor inválido!");
} else if (valorSaque > saldo) {
  console.log("Saldo insuficiente!");
} else {
  console.log(`Saque de R$${valorSaque} realizado.`);
}

// Operador ternário — if/else em uma linha
const status = saldo > 0 ? "Positivo" : "Zerado";
console.log(status); // "Positivo"

// Guard clause — retornar cedo para evitar aninhamento
function sacar(valor, saldo) {
  if (valor <= 0) return "Valor inválido";
  if (valor > saldo) return "Saldo insuficiente";
  return `Saque de R$${valor} realizado`;
}
// Mais limpo que if/else/if/else aninhado!
```

O padrão de **guard clause** (retornar cedo) será usado constantemente nos métodos e setters das nossas classes. Ele evita aninhamento excessivo e deixa o código mais legível.

Slide 10 de 45

Switch Case

Quando há muitas condições sobre o mesmo valor

```
const tipoPagamento = "pix";

switch (tipoPagamento) {
  case "cartao":
    console.log("Processando cartão...");
    break;
  case "pix":
    console.log("Gerando QR Code...");
    break;
  case "boleto":
    console.log("Gerando boleto...");
    break;
  default:
    console.log("Tipo de pagamento desconhecido.");
}
// Output: "Gerando QR Code..."

// ⚠️ CUIDADO: sem o break, o código "cai" para o próximo case!
// Isso é um bug muito comum em iniciantes.
```

O `switch` é útil quando comparamos **o mesmo valor** contra várias opções. Em OOP, pode aparecer em métodos que tratam diferentes tipos ou estados. Mas muitas vezes, polimorfismo substitui o `switch` de forma mais elegante — veremos isso na aula 4.

Slide 11 de 45

Estruturas de Repetição — for e while

Loops tradicionais

```
// for clássico - quando sabemos quantas vezes repetir
for (let i = 0; i < 5; i++) {
  console.log(`Iteração ${i}`); // 0, 1, 2, 3, 4
}

// while - quando NÃO sabemos
quantas vezes (depende de condição)
let tentativas = 0;
let acertou = false;

while (!acertou && tentativas < 3) {
  tentativas++;
  // simula uma tentativa
  if (tentativas === 3) acertou = true;
  console.log(`Tentativa ${tentativas}`);
}

// do...while - executa pelo menos 1 vez antes de checar
let entrada;
do {
  entrada = "algum valor"; // simulando input
  console.log("Processando...");
} while (entrada === "");
```

No módulo de OOP, o `while` pode aparecer em lógica de jogos (jogo da forca: enquanto não acertou e não perdeu). O `for` clássico é menos usado — preferimos métodos de array como `forEach`, `map` e `filter`.

Slide 12 de 45

for...of e for...in

Iterando sobre arrays e objetos

```
// for...of - itera sobre os VALORES
de um iterável (arrays, strings, Sets)
const frutas = ["maçã", "banana", "uva"];

for (const fruta of frutas) {
  console.log(fruta); // "maçã", "banana", "uva"
}

// Funciona com strings também
for (const letra of "OOP") {
  console.log(letra); // "O", "O", "P"
}

// for...in - itera sobre as CHAVES de um objeto
const pessoa = { nome: "Ana", idade: 25, cidade: "SP" };

for (const chave in pessoa) {
  console.log(`${chave}: ${pessoa[chave]}`);
  // "nome: Ana", "idade: 25", "cidade: SP"
}

// ⚠️ NÃO use for...in em arrays - use for...of ou forEach
// for...in em array itera sobre os
ÍNDICES como strings, não os valores
```

Loop	Itera sobre	Usar com
for...of	Valores	Arrays, strings, Sets, Maps
for...in	Chaves (propriedades)	Objetos literais

Slide 13 de 45

Verificação — Loops

for...of vs for...in

O que imprime o código: `for (const x of [10, 20, 30]) { console.log(x); }`?

0, 1, 2 (os índices)

✓ 10, 20, 30 (os valores) (Resposta Correta)

Erro — arrays não suportam for...of

"0", "1", "2" (os índices como string)

Explicação: Isso! `for...of` itera sobre os VALORES. Para iterar sobre índices, seria `for...in` (mas evite em arrays).

Slide 14 de 45

Funções — Declaração e Expressão

As formas tradicionais de criar funções

Funções são blocos de código reutilizáveis. Existem várias formas de declará-las em JavaScript. Entender as diferenças é essencial porque no OOP usaremos métodos (que são funções dentro de classes).

```
// 1. Declaração de função (function declaration)
// Sofre hoisting — pode ser chamada ANTES de ser declarada
function somar(a, b) {
  return a + b;
}
console.log(somar(2, 3)); // 5

// 2. Expressão de função (function expression)
// NÃO sofre hoisting — só pode ser
usada após a linha onde foi definida
const subtrair = function(a, b) {
  return a - b;
};
console.log(subtrair(10, 4)); // 6

// 3. Parâmetros com valores padrão
function criarUsuario(nome = "Anônimo", idade = 0) {
  return { nome, idade };
}
console.log(criarUsuario());
// { nome: "Anônimo", idade: 0 }
console.log(criarUsuario("Ana",
25)); // { nome: "Ana", idade: 25 }
```

Valores padrão serão usados em TODOS os constructors das nossas classes. O padrão `constructor(nome = "Sem nome", preco = 0)` vem diretamente deste conceito.

Slide 15 de 45

Arrow Functions

A sintaxe moderna e suas diferenças

```
// Arrow function — sintaxe curta
const multiplicar = (a, b) => {
  return a * b;
};

// Se o corpo tem UMA expressão, pode omitir {} e return
const dobrar = (n) => n * 2;

// Se tem UM parâmetro, pode omitir os parênteses
const quadrado = n => n ** 2;

// Com arrays — arrow functions brilham aqui
const numeros = [1, 2, 3, 4, 5];
const dobrados = numeros.map(n =>
n * 2);       // [2, 4, 6, 8, 10]
const pares = numeros.filter(n => n % 2 === 0);  // [2, 4]
const soma = numeros.reduce((acc, n) => acc + n, 0); // 15
```

Diferença crucial com `this`: arrow functions NÃO têm seu próprio `this` — elas herdam o `this` do contexto onde foram criadas. Por isso, dentro de classes usamos a sintaxe normal de método (`metodo() { }`) e NÃO arrow functions.

```
// ⚠️ Armadilha com this em arrow functions:
class Exemplo {
  constructor() {
    this.valor = 42;
  }

  // ✅ Método normal - this funciona
  metodoNormal() {
    console.log(this.valor); // 42
  }

  // ❌ Arrow como método - NÃO faça isso em classes
  // metodoArrow = () => {
  //   console.log(this.valor);
  // }
  // Funciona neste caso, mas tem
comportamento diferente com herança
}
```

Slide 16 de 45

Funções — return e Escopo

O que retorna e o que fica preso dentro da função

```
// return encerra a função e devolve um valor
function verificarIdade(idade) {
  if (idade < 0) return "Idade
inválida";  // sai aqui se for negativo
  if (idade < 18) return "Menor de idade";
  return "Maior de idade";
  // Código aqui NUNCA executa (após return)
}

// Função sem return explícito retorna undefined
function exibirMensagem(msg) {
  console.log(msg);
  // return undefined (implícito)
}

const resultado = exibirMensagem("Olá"); // imprime "Olá"
console.log(resultado); // undefined
```

```
// ESCOPO - variáveis declaradas dentro de {} não existem fora
function exemplo() {
  const dentroFuncao = "Existo só aqui";
  if (true) {
    const dentroIf = "Existo só neste bloco";
    console.log(dentroFuncao); //
✅ acessa a variável da função
  }
  // console.log(dentroIf); // ❌ ReferenceError!
}
// console.log(dentroFuncao); // ❌ ReferenceError!

// Escopo é importante em OOP:
// Atributos (this.x) vivem no OBJETO
- acessíveis em qualquer método
// Variáveis locais (const x) vivem no
MÉTODO - morrem quando o método termina
```

Em OOP, essa distinção será vital: `this.nome` (atributo do objeto,

persiste) vs `const temp` dentro de um método (variável local, temporária).

Slide 17 de 45

Verificação — Funções

Return e valores padrão

Qual o output? `function f(a = 5, b = 10) { return a + b; } console.log(f(3));`

15 (5 + 10)

✓ 13 (3 + 10) (Resposta Correta)

8 (3 + 5)

NaN

Explicação: Correto! O primeiro argumento (3) substitui o valor padrão de `a`. O segundo parâmetro `b` não recebeu argumento, então usa o padrão 10. Resultado: 3 + 10 = 13.

Slide 18 de 45

Arrays — Criação e Acesso

A estrutura de dados mais usada em JavaScript

Arrays armazenam listas ordenadas de valores. Em OOP, usaremos arrays constantemente: listas de tarefas, itens do carrinho, letras chutadas, histórico de partidas...

```
// Criação
const frutas = ["maçã", "banana", "uva"];
const numeros = [1, 2, 3, 4, 5];
const misturado = ["Ana", 25, true,
null]; // JS permite tipos mistos
const vazio = [];

// Acesso por índice (começa em 0)
console.log(frutas[0]); // "maçã"
console.log(frutas[2]); // "uva"
console.log(frutas[5]); // undefined (não existe)

// Propriedade length
console.log(frutas.length); // 3

// Último elemento
console.log(frutas[frutas.length - 1]); // "uva"

// Modificar elemento
frutas[1] = "morango";
console.log(frutas); // ["maçã", "morango", "uva"]

// Verificar se é array
console.log(Array.isArray(frutas)); // true
console.log(Array.isArray("texto")); // false
```

Slide 19 de 45

Arrays — Métodos que Modificam (Mutáveis)

push, pop, shift, unshift, splice

Estes métodos **alteram o array original**. Importante saber disso porque em OOP, quando um atributo é um array, esses métodos modificam o estado do objeto.

```
const itens = ["A", "B", "C"];

// push - adiciona ao FINAL
itens.push("D");
console.log(itens); // ["A", "B", "C", "D"]

// pop - remove do FINAL e retorna o removido
const ultimo = itens.pop();
console.log(ultimo); // "D"
console.log(itens);  // ["A", "B", "C"]

// unshift - adiciona ao INÍCIO
itens.unshift("Z");
console.log(itens); // ["Z", "A", "B", "C"]

// shift - remove do INÍCIO e retorna o removido
const primeiro = itens.shift();
console.log(primeiro); // "Z"
console.log(itens);    // ["A", "B", "C"]

// splice - remove/insere em posição específica
// splice(índice, quantos_remover, ...itens_inserir)
itens.splice(1, 1);         // remove 1 elemento na posição 1
console.log(itens);         // ["A", "C"]

itens.splice(1, 0, "B");    // insere
"B" na posição 1 sem remover nada
console.log(itens);         // ["A", "B", "C"]
```

Slide 20 de 45

Arrays — Métodos que NÃO Modificam (Imutáveis)

map, filter, find, reduce, forEach, includes

Estes métodos **retornam algo novo** sem alterar o array original. São os mais usados em OOP moderno e praticamente obrigatórios para o módulo.

```
const produtos = [
  { nome: "Café", preco: 12.90 },
  { nome: "Açúcar", preco: 5.50 },
  { nome: "Leite", preco: 7.80 },
  { nome: "Pão", preco: 8.00 },
];

// forEach — executa uma função
para cada item (não retorna nada)
produtos.forEach(p => console.log(p.nome));
// "Café", "Açúcar", "Leite", "Pão"

// map — transforma cada item e retorna NOVO array
const nomes = produtos.map(p => p.nome);
console.log(nomes); // ["Café", "Açúcar", "Leite", "Pão"]

// filter — filtra itens e retorna NOVO array
const baratos = produtos.filter(p => p.preco < 8);
console.log(baratos); // [{ nome:
"Açúcar"... }, { nome: "Leite"... }]

// find — retorna o PRIMEIRO item que
satisfaz a condição (ou undefined)
const cafe = produtos.find(p => p.nome === "Café");
console.log(cafe); // { nome: "Café", preco: 12.90 }

// includes — verifica se o array
contém um valor (retorna boolean)
const numeros = [1, 2, 3, 4, 5];
console.log(numeros.includes(3)); // true
console.log(numeros.includes(9)); // false
```

Slide 21 de 45

Arrays — reduce e sort

Os dois métodos mais poderosos

```
// reduce — acumula todos os itens em um único valor
const precos = [12.90, 5.50, 7.80, 8.00];

const total = precos.reduce((acumulador, precoAtual) => {
  return acumulador + precoAtual;
}, 0); // 0 é o valor inicial do acumulador

console.log(total); // 34.20

// Versão curta:
const total2 = precos.reduce((acc, p) => acc + p, 0);

// Exemplo real: calcular total de um carrinho
const carrinho = [
  { nome: "Café", preco: 12.90, qtd: 2 },
  { nome: "Pão", preco: 8.00, qtd: 1 },
];

const totalCarrinho = carrinho.reduce((acc, item) => {
  return acc + (item.preco * item.qtd);
}, 0);
console.log(totalCarrinho); // 33.80
```

```
// sort – ordena o array (⚠️ MODIFICA o original!)
const nomes = ["Carlos", "Ana", "Bia"];
nomes.sort();
console.log(nomes); // ["Ana", "Bia", "Carlos"]

// ⚠️ Armadilha com números:
const nums = [10, 1, 21, 2];
nums.sort();
console.log(nums); // [1, 10, 2,
21] ← ERRADO! Ordena como string!

// Correção: passar função de comparação
nums.sort((a, b) => a - b); // crescente
console.log(nums); // [1, 2, 10, 21] ✅

nums.sort((a, b) => b - a); // decrescente
console.log(nums); // [21, 10, 2, 1] ✅

// Ordenar objetos por propriedade:
const pessoas = [
  { nome: "Carlos", idade: 30 },
  { nome: "Ana", idade: 25 },
];
pessoas.sort((a, b) => a.idade -
b.idade); // por idade crescente
```

Slide 22 de 45

Verificação — Métodos de Array

filter, find e reduce

Qual a diferença entre `filter` e `find`?

Nenhuma — fazem a mesma coisa

✓ filter retorna um ARRAY com todos que satisfazem; find retorna apenas o PRIMEIRO item (Resposta Correta)

find retorna um array; filter retorna um item

filter modifica o original; find não

Explicação: Exato! `filter` retorna um novo array com TODOS os itens que passam no teste. `find` retorna apenas o PRIMEIRO item que passa (ou undefined se nenhum passar).

Slide 23 de 45

Objetos Literais — Criação e Acesso

A base para entender classes

Objetos literais são coleções de pares **chave: valor**. Eles são a base para entender classes — uma classe é basicamente um objeto com estrutura garantida e métodos embutidos.

```
// Criação
const pessoa = {
  nome: "Ana",
  idade: 25,
  cidade: "Foz do Iguaçu",
  ativo: true,
};

// Acesso — notação de ponto (mais comum)
console.log(pessoa.nome);   // "Ana"
console.log(pessoa.idade);  // 25

// Acesso — notação de colchetes (útil com variáveis)
const campo = "cidade";
console.log(pessoa[campo]);  // "Foz do Iguaçu"
// console.log(pessoa.campo); //
undefined! Busca literalmente "campo"

// Modificar
pessoa.idade = 26;

// Adicionar nova propriedade
pessoa.email = "ana@email.com";

// Deletar propriedade
delete pessoa.ativo;

// Verificar se propriedade existe
console.log("nome" in pessoa);  // true
console.log("cpf" in pessoa);   // false
```

Slide 24 de 45

Objetos — Métodos e Shorthand

Funções dentro de objetos e atalhos modernos

```javascript
// Objeto com método
const calculadora = {
  // Método — função que vive dentro do objeto
  somar(a, b) {
    return a + b;
  },
  subtrair(a, b) {
    return a - b;
  },
};

console.log(calculadora.somar(5, 3));    // 8
console.log(calculadora.subtrair(10, 4)); // 6

// this em objetos literais
const usuario = {
  nome: "Carlos",
  idade: 30,
  apresentar() {
    // this se refere ao próprio objeto
    console.log(`Sou ${this.nome}, tenho ${this.idade} anos.`);
  },
};

usuario.apresentar(); // "Sou Carlos, tenho 30 anos."
```

```
// Shorthand — atalhos modernos para objetos

// 1. Property shorthand — quando a
variável tem o mesmo nome da chave
const nome = "Ana";
const idade = 25;

// ❌ Sem shorthand (repetitivo)
const pessoa1 = { nome: nome, idade: idade };

// ✅ Com shorthand (limpo)
const pessoa2 = { nome, idade };
// Ambos geram: { nome: "Ana", idade: 25 }

// 2. Computed property names — chave dinâmica
const campo = "email";
const obj = { [campo]: "ana@mail.com" };
console.log(obj.email); // "ana@mail.com"
```

Métodos dentro de objetos literais são o "ancestral" dos métodos de classe. A diferença: no objeto literal, cada objeto precisa ter seus métodos definidos individualmente. Na classe, os métodos são definidos uma vez e compartilhados por todas as instâncias.

Slide 25 de 45

Objetos — Object.keys, values e entries

Extraindo informações de objetos

```
const produto = {
  nome: "Notebook",
  preco: 3500,
  estoque: 10,
  marca: "Dell",
};

// Object.keys - retorna array com as CHAVES
const chaves = Object.keys(produto);
console.log(chaves); // ["nome", "preco", "estoque", "marca"]

// Object.values - retorna array com os VALORES
const valores = Object.values(produto);
console.log(valores); // ["Notebook", 3500, 10, "Dell"]

// Object.entries - retorna array de pares [chave, valor]
const pares = Object.entries(produto);
console.log(pares);
// [["nome", "Notebook"], ["preco",
3500], ["estoque", 10], ["marca", "Dell"]]

// Útil para iterar:
Object.entries(produto).forEach(([chave, valor]) => {
  console.log(`${chave}: ${valor}`);
});
// "nome: Notebook"
// "preco: 3500"
// "estoque: 10"
// "marca: Dell"
```

Esses métodos são úteis em OOP quando precisamos inspecionar objetos, criar representações em string, ou comparar propriedades entre instâncias.

Slide 26 de 45

Verificação — Objetos

Testando o conhecimento de objetos

Dado `const obj = { a: 1, b: 2, c: 3 }`, o que retorna `Object.keys(obj)`?

[1, 2, 3]

✓ ["a", "b", "c"] (Resposta Correta)

{ a: 1, b: 2, c: 3 }

[["a", 1], ["b", 2], ["c", 3]]

Explicação: Correto! `Object.keys` retorna um array com as CHAVES (nomes das propriedades) do objeto. Para valores, use `Object.values`. Para pares, use `Object.entries`.

Slide 27 de 45

Template Literals

Strings poderosas com crases

Template literals (com crases `` ` ` ``) permitem interpolar variáveis e expressões diretamente dentro de strings. Serão usados em praticamente **todos** os métodos das nossas classes.

```
const nome = "Ana";
const idade = 25;
const preco = 49.90;

// ❌ Concatenação antiga - confusa com muitas variáveis
console.log("Olá, " + nome + "! Você tem " + idade + " anos.");

// ✅ Template literal - limpo e legível
console.log(`Olá, ${nome}! Você tem ${idade} anos.`);

// Aceita EXPRESSÕES dentro de ${}
console.log(`Preço com desconto:
R$${(preco * 0.9).toFixed(2)}`);
// "Preço com desconto: R$44.91"

// Aceita chamadas de função
console.log(`Nome em maiúsculo: ${nome.toUpperCase()}`);
// "Nome em maiúsculo: ANA"

// Aceita operador ternário
const status = `Usuário ${idade >=
18 ? "maior" : "menor"} de idade`;
console.log(status); // "Usuário maior de idade"

// Strings multilinha
const html = `
  <div>
    <h1>${nome}</h1>
    <p>Idade: ${idade}</p>
  </div>
`;
```

Slide 28 de 45

Desestruturação

Extraindo valores de objetos e arrays de forma limpa

```
// DESESTRUTURAÇÃO DE OBJETOS
const pessoa = { nome: "Ana", idade: 25, cidade: "Foz" };

// ❌ Sem desestruturação
const nome1 = pessoa.nome;
const idade1 = pessoa.idade;

// ✅ Com desestruturação - extrai
múltiplas propriedades de uma vez
const { nome, idade, cidade } = pessoa;
console.log(nome);   // "Ana"
console.log(idade);  // 25
console.log(cidade); // "Foz"

// Com renomeação
const { nome: nomeUsuario, idade: idadeUsuario } = pessoa;
console.log(nomeUsuario); // "Ana"

// Com valor padrão
const { email = "não informado" } = pessoa;
console.log(email); // "não informado"
```

```
// DESESTRUTURAÇÃO DE ARRAYS
const coordenadas = [25.5, -54.5, 180];

const [latitude, longitude, altitude] = coordenadas;
console.log(latitude);  // 25.5
console.log(longitude); // -54.5

// Ignorar elementos com vírgula
const [primeiro, , terceiro] = [10, 20, 30];
console.log(primeiro); // 10
console.log(terceiro); // 30

// DESESTRUTURAÇÃO EM PARÂMETROS DE FUNÇÃO
function exibirProduto({ nome, preco }) {
  console.log(`${nome}: R$${preco.toFixed(2)}`);
}

exibirProduto({ nome: "Café", preco: 12.90 });
// "Café: R$12.90"
```

Desestruturação em parâmetros é muito comum em código moderno. Em OOP, aparece quando métodos recebem objetos complexos e precisam extrair apenas algumas propriedades.

Slide 29 de 45

Spread e Rest Operators

Os três pontinhos (...) que fazem tudo

```
// SPREAD (...) - "espalha" elementos

// Copiar array (cópia rasa)
const original = [1, 2, 3];
const copia = [...original];
copia.push(4);
console.log(original); // [1, 2, 3] - inalterado!
console.log(copia);    // [1, 2, 3, 4]

// Juntar arrays
const a = [1, 2];
const b = [3, 4];
const junto = [...a, ...b];
console.log(junto); // [1, 2, 3, 4]

// Copiar objeto (cópia rasa)
const pessoa = { nome: "Ana", idade: 25 };
const clone = { ...pessoa };
clone.idade = 30;
console.log(pessoa.idade); // 25 - inalterado!

// Mesclar objetos (merge)
const padrao = { cor: "azul", tamanho: "M" };
const customizado = { tamanho: "G", preco: 50 };
const resultado = { ...padrao, ...customizado };
console.log(resultado); // { cor:
"azul", tamanho: "G", preco: 50 }
// tamanho "G" sobrescreveu "M" porque veio depois
```

```
// REST (...) - "coleta" elementos restantes

// Em funções - coleta argumentos extras
function somar(primeiro, ...restante) {
  console.log(primeiro);  // 1
  console.log(restante);  // [2, 3, 4, 5]
  return restante.reduce((acc, n) => acc + n, primeiro);
}
console.log(somar(1, 2, 3, 4, 5)); // 15

// Em desestruturação - coleta o resto
const [cabeca, ...cauda] = [10, 20, 30, 40];
console.log(cabeca); // 10
console.log(cauda);  // [20, 30, 40]

const { nome, ...resto } = { nome:
"Ana", idade: 25, cidade: "Foz" };
console.log(nome);  // "Ana"
console.log(resto);  // { idade: 25, cidade: "Foz" }
```

O spread `[...array]` será usado em OOP para retornar **cópias** de arrays internos nos getters, protegendo o encapsulamento. Se retornarmos a referência direta, código externo pode modificar o array interno.

Slide 30 de 45

Verificação — Spread

Cópia vs referência com spread

Dado `const a = [1,2,3]; const b = [...a]; b.push(4);`, qual o valor de `a`?

[1, 2, 3, 4]

✓ [1, 2, 3] (Resposta Correta)

Erro — arrays const não podem ser modificados

[4, 1, 2, 3]

Explicação: Correto! O spread `[...a]` cria uma CÓPIA independente. Modificar `b` não afeta `a`. Esse padrão será muito usado nos getters das classes para proteger arrays internos.

Slide 31 de 45

Tratamento de Erros — try/catch e throw

Como lidar com erros de forma controlada

Em OOP, lançar e capturar erros é a forma profissional de lidar com dados inválidos. Os setters das nossas classes usarão `throw` para rejeitar valores ruins.

```
// throw — LANÇAR um erro intencionalmente
function dividir(a, b) {
  if (b === 0) {
    throw new Error("Divisão por zero não é permitida!");
  }
  return a / b;
}

// try/catch — CAPTURAR o erro sem crashar o programa
try {
  const resultado = dividir(10, 0);
  console.log(resultado); // nunca executa
} catch (erro) {
  console.log(erro.message); //
"Divisão por zero não é permitida!"
}

console.log("Programa continua rodando!"); // ✅ não crashou
```

```
// Exemplo real — validação de dados
function criarUsuario(nome, email) {
  if (!nome || !nome.trim()) {
    throw new Error("Nome não pode ser vazio.");
  }
  if (!email || !email.includes("@")) {
    throw new Error("Email inválido.");
  }
  return { nome: nome.trim(), email: email.toLowerCase() };
}

// Usando:
try {
  const user = criarUsuario("", "ana@email.com");
} catch (e) {
  console.log(e.message); // "Nome não pode ser vazio."
}

try {
  const user = criarUsuario("Ana", "sem-arroba");
} catch (e) {
  console.log(e.message); // "Email inválido."
}

try {
  const user = criarUsuario("Ana", "ana@email.com");
  console.log(user); // { nome:
"Ana", email: "ana@email.com" } ✅
} catch (e) {
  // Não entra aqui — tudo válido
}
```

Na aula 3 de OOP (Encapsulamento), os setters usarão `throw new Error()` para impedir valores inválidos: preço negativo, email sem @, senha curta demais. O `try/catch` fica no código que CHAMA, não dentro da classe.

Métodos Úteis de String

Os métodos que mais usaremos no módulo

```
const texto = "  JavaScript é Incrível!  ";

// Limpeza
console.log(texto.trim());       //
"JavaScript é Incrível!" (remove espaços)
console.log(texto.trimStart());  // "JavaScript é Incrível!  "
console.log(texto.trimEnd());    // "  JavaScript é Incrível!"

// Maiúsculas e minúsculas
console.log("hello".toUpperCase()); // "HELLO"
console.log("HELLO".toLowerCase()); // "hello"

// Busca
console.log("email@teste.com".includes("@")); // true
console.log("Olá mundo".startsWith("Olá"));   // true
console.log("arquivo.pdf".endsWith(".pdf"));  // true
console.log("abcabc".indexOf("b"));
// 1 (primeiro índice)

// Divisão e junção
console.log("a,b,c".split(","));     // ["a", "b", "c"]
console.log(["a","b","c"].join("-")); // "a-b-c"

// Substituição
console.log("Olá mundo".replace("mundo", "JS")); // "Olá JS"

// Caractere por posição
console.log("Hello".charAt(0)); // "H"

// Substring
console.log("JavaScript".slice(0, 4)); // "Java"
console.log("JavaScript".slice(4));    // "Script"
```

No módulo de OOP: `trim()` será usado em validações de setter (nome não pode ser só espaços), `includes("@")` para validar email, `toUpperCase()` no jogo

da forca para normalizar letras, `split/join` para manipular a palavra exibida.

Slide 33 de 45

Métodos Úteis de Number e Math

Formatação e cálculos

```
// toFixed — formatar casas decimais (retorna STRING!)
const preco = 19.9;
console.log(preco.toFixed(2));  // "19.90"
console.log((10/3).toFixed(2)); // "3.33"

// Converter string para número
console.log(parseInt("42px"));     // 42 (ignora o texto)
console.log(parseFloat("3.14abc")); // 3.14
console.log(Number("42"));          // 42
console.log(Number("abc"));         // NaN

// Verificar se é número válido
console.log(Number.isNaN(NaN));       // true
console.log(Number.isNaN(42));        // false
console.log(Number.isInteger(42));    // true
console.log(Number.isInteger(42.5));  // false
```

```
// Math — métodos estáticos (spoiler: veremos static na aula 4!)
console.log(Math.PI);           // 3.141592653589793
console.log(Math.round(4.5));   // 5 (arredonda)
console.log(Math.floor(4.9));   // 4 (arredonda para baixo)
console.log(Math.ceil(4.1));    // 5 (arredonda para cima)
console.log(Math.max(1, 5, 3)); // 5
console.log(Math.min(1, 5, 3)); // 1
console.log(Math.abs(-7));      // 7 (valor absoluto)
console.log(Math.pow(2, 3));    // 8 (ou 2 ** 3)
console.log(Math.sqrt(16));     // 4

// Número aleatório entre min e max (inteiro)
function randomEntre(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
console.log(randomEntre(1, 10)); // número entre 1 e 10
```

No módulo: `toFixed(2)` para exibir preços, `Math.PI` para áreas de círculos, `Math.random()` para sortear palavras no jogo da forca, `Math.floor/ceil` para cálculos diversos.

Slide 34 de 45

Set — A Estrutura Sem Duplicatas

Perfeito para o jogo da forca

O `Set` é uma coleção que **não permite valores duplicados**. Se tentar adicionar algo que já existe, ele simplesmente ignora. Usaremos no jogo da forca para armazenar letras já chutadas.

```javascript
// Criação
const letras = new Set();

// Adicionar
letras.add("A");
letras.add("B");
letras.add("A"); // ignorado! Já existe
console.log(letras); // Set { "A", "B" }

// Verificar se contém
console.log(letras.has("A")); // true
console.log(letras.has("C")); // false

// Tamanho
console.log(letras.size); // 2 (não 3!)

// Remover
letras.delete("A");
console.log(letras.has("A")); // false

// Iterar
letras.add("X");
letras.add("Y");
letras.forEach(l => console.log(l)); // "B", "X", "Y"

// Converter para array
const arr = [...letras]; // ["B", "X", "Y"]

// Uso clássico: remover duplicatas de um array
const numeros = [1, 2, 2, 3, 3, 3];
const unicos = [...new Set(numeros)];
console.log(unicos); // [1, 2, 3]
```

Slide 35 de 45

Date — Trabalhando com Datas

Básico de datas para validações

```
// Criar data atual
const agora = new Date();
console.log(agora); // ex: 2026-04-06T14:30:00.000Z

// Criar data específica
const natal = new Date(2026, 11, 25);
// ⚠️ mês começa em 0! (11 = dezembro)
const dataString = new Date("2026-12-25");

// Extrair partes
console.log(agora.getFullYear()); // 2026
console.log(agora.getMonth());    // 0-11 (⚠️ janeiro = 0!)
console.log(agora.getDate());     // 1-31
console.log(agora.getHours());    // 0-23
console.log(agora.getDay());      // 0-6 (0 = domingo)

// Comparar datas
const data1 = new Date("2026-01-01");
const data2 = new Date("2026-06-01");
console.log(data1 < data2);  // true
console.log(data1 > data2);  // false

// Diferença em dias
const diff = data2 - data1; // diferença em milissegundos
const dias = Math.floor(diff / (1000 * 60 * 60 * 24));
console.log(dias); // 151

// Timestamp
console.log(Date.now()); // milissegundos desde 01/01/1970
```

No módulo de OOP: `new Date()` será usado em atributos como `dataCriacao` de tarefas, `dataValidade` de produtos, e para calcular se um produto está vencido.

Slide 36 de 45

Verificação — Set e Date

Testando Set

Qual o `size` do Set após executar: `const s = new Set(); s.add(1); s.add(2); s.add(1); s.add(3); s.add(2);`?

5

✓ 3 (Resposta Correta)

2

Erro — Set não tem size

Explicação: Correto! O Set ignora duplicatas. Valores únicos são 1, 2 e 3, então `size` é 3. Mesmo adicionando 5 vezes, só 3 valores distintos entram.

Slide 37 de 45

Console — Ferramentas de Debug

Além do console.log

```
// console.log — o básico
console.log("Mensagem");
console.log("Valor:", 42, "e", true); // múltiplos argumentos

// console.table — exibe objetos/arrays em formato de tabela
const produtos = [
  { nome: "Café", preco: 12.90 },
  { nome: "Açúcar", preco: 5.50 },
];
console.table(produtos);
// Mostra uma tabela formatada no console!

// console.warn — aviso (aparece em amarelo)
console.warn("Atenção: valor muito alto!");

// console.error — erro (aparece em vermelho)
console.error("Erro: operação falhou!");

// console.group / groupEnd — agrupar logs
console.group("Dados do usuário");
console.log("Nome: Ana");
console.log("Idade: 25");
console.groupEnd();

// console.time / timeEnd — medir tempo de execução
console.time("operação");
// ... código ...
console.timeEnd("operação"); // "operação: 0.123ms"
```

`console.table` é especialmente útil no módulo de OOP para visualizar arrays de objetos (lista de tarefas, carrinho, produtos). Sempre prefira `console.table` quando estiver inspecionando arrays de objetos.

Slide 38 de 45

JSON — Serialização de Dados

Converter objetos para texto e vice-versa

JSON (JavaScript Object Notation) é o formato universal para trocar dados. Usaremos no módulo de OOP para salvar dados no `localStorage` e no futuro para comunicação com APIs.

```
const produto = {
  nome: "Café",
  preco: 12.90,
  tags: ["bebida", "quente"],
};

// Objeto → String JSON
const json = JSON.stringify(produto);
console.log(json);
// '{"nome":"Café","preco":12.9,"tags":["bebida","quente"]}'
console.log(typeof json); // "string"

// String JSON → Objeto
const obj = JSON.parse(json);
console.log(obj.nome); // "Café"
console.log(typeof obj); // "object"

// Formatado (útil para debug)
console.log(JSON.stringify(produto, null, 2));
// {
//   "nome": "Café",
//   "preco": 12.9,
//   "tags": [
//     "bebida",
//     "quente"
//   ]
// }
```

Limitação importante para OOP: `JSON.stringify` salva apenas DADOS (atributos). **Métodos são perdidos** na conversão. Por isso, ao carregar dados do localStorage, precisamos RECRIAR instâncias usando o constructor da classe — não basta dar `JSON.parse`.

Slide 39 de 45

Verificação — JSON

Serialização e suas limitações

O que acontece com os MÉTODOS de um objeto quando fazemos `JSON.stringify(objeto)`?

São convertidos para string normalmente

✓ São ignorados e perdidos — JSON só salva dados (Resposta Correta)

Causam um erro

São convertidos para null

Explicação: Correto! JSON não suporta funções/métodos. Apenas dados (strings, numbers, booleans, arrays, objetos, null) são serializados. Métodos são simplesmente ignorados.

Slide 40 de 45

Manipulação do DOM — Básico

Conectando JavaScript ao HTML

O DOM (Document Object Model) é a representação do HTML como objetos JavaScript. No módulo de OOP, usaremos o DOM para conectar nossas classes à interface visual.

```
// SELECIONAR elementos
const titulo =
document.getElementById("titulo");          // por ID
const botoes = document.querySelectorAll(".btn");
// todos com classe
const primeiro = document.querySelector(".lista
li");       // primeiro que bate

// LER conteúdo
console.log(titulo.textContent);   // texto do elemento
console.log(titulo.innerHTML);     // HTML interno

// ALTERAR conteúdo
titulo.textContent = "Novo Título";
titulo.innerHTML = "<em>Itálico</em>";

// ALTERAR estilos
titulo.style.color = "red";
titulo.style.display = "none"; // esconder
titulo.style.display = "block"; // mostrar

// CLASSES CSS
titulo.classList.add("ativo");     // adicionar classe
titulo.classList.remove("ativo");  // remover classe
titulo.classList.toggle("ativo");  // alternar (add/remove)
```

Slide 41 de 45

DOM — Criar e Remover Elementos

Manipulação dinâmica

```
// CRIAR elemento
const novaDiv = document.createElement("div");
novaDiv.textContent = "Sou um elemento novo!";
novaDiv.classList.add("card");

// ADICIONAR ao documento
const container = document.getElementById("container");
container.appendChild(novaDiv);

// CRIAR com innerHTML (mais rápido para HTML complexo)
const lista = document.getElementById("lista");
lista.innerHTML = `
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
`;
// ⚠️ innerHTML SUBSTITUI todo o conteúdo interno

// ADICIONAR sem apagar o existente
lista.innerHTML += `<li>Item 4</li>`;

// REMOVER elemento
const item = document.querySelector("li");
item.remove();

// LIMPAR container inteiro
container.innerHTML = ""; // remove todos os filhos
```

No módulo de OOP, o padrão será: **limpar o container** (`innerHTML = ""`) e **recriar todos os elementos** a cada mudança de estado. Isso é simples e garante que a tela sempre reflete os dados atuais.

Slide 42 de 45

DOM — Eventos

Respondendo a ações do usuário

```
// addEventListener - forma moderna de escutar eventos
const botao = document.getElementById("btn-salvar");

botao.addEventListener("click", function() {
  console.log("Botão clicado!");
});

// Com arrow function
botao.addEventListener("click", () => {
  console.log("Clicou!");
});

// Eventos comuns:
// click      - clique do mouse
// input      - valor do input mudou (a cada tecla)
// change     - valor mudou E perdeu foco
// submit     - formulário enviado
// keydown    - tecla pressionada
// keyup      - tecla solta
```

```javascript
// FORMULÁRIO — capturar valores
const form = document.getElementById("form-tarefa");

form.addEventListener("submit", (event) => {
  event.preventDefault(); // impede o reload da página!

  const titulo = document.getElementById("input-titulo").value;
  const descricao = document.getElementById("input-desc").value;

  console.log(`Título: ${titulo}, Descrição: ${descricao}`);

  // Limpar inputs após capturar
  document.getElementById("input-titulo").value = "";
  document.getElementById("input-desc").value = "";
});

// INPUT — reagir em tempo real
const campoBusca = document.getElementById("busca");
campoBusca.addEventListener("input", (event) => {
  const termo = event.target.value;
  console.log(`Buscando: ${termo}`);
});
```

No módulo de OOP: o `submit` do formulário vai chamar métodos das nossas classes (ex: `gerenciador.adicionarTarefa(titulo, desc)`). O `event.preventDefault()` é **obrigatório** em formulários para evitar que a página recarregue.

Slide 43 de 45

Verificação — DOM

Eventos e manipulação

Por que usamos `event.preventDefault()` no evento submit de um formulário?

Para impedir que o formulário seja exibido

✓ Para impedir que a página recarregue ao enviar o formulário (Resposta Correta)

Para impedir que o JavaScript execute

Para limpar os campos do formulário

Explicação: Correto! O comportamento padrão do submit é recarregar a página (enviar dados para o servidor). Como trabalhamos com JavaScript no frontend, usamos `preventDefault()` para evitar o reload e processar os dados com nossas classes.

Slide 44 de 45

localStorage — Persistência no Navegador

Salvando dados entre sessões

```javascript
// SALVAR — sempre como string
localStorage.setItem("nome", "Ana");
localStorage.setItem("idade", "25"); // número vira string

// Para objetos/arrays: converter com JSON.stringify
const tarefas = [
  { titulo: "Estudar JS", concluida: false },
  { titulo: "Fazer exercício", concluida: true },
];
localStorage.setItem("tarefas", JSON.stringify(tarefas));

// LER — sempre retorna string (ou null se não existir)
const nome = localStorage.getItem("nome");
console.log(nome); // "Ana"

// Para objetos/arrays: converter com JSON.parse
const dadosSalvos = localStorage.getItem("tarefas");
const listaTarefas = JSON.parse(dadosSalvos);
console.log(listaTarefas); // [{ titulo: "Estudar JS"... }, ...]

// VERIFICAR se existe
const dados = localStorage.getItem("chave_inexistente");
console.log(dados); // null

// Padrão seguro:
const itens = JSON.parse(localStorage.getItem("itens") || "[]");

// REMOVER um item
localStorage.removeItem("nome");

// LIMPAR tudo
localStorage.clear();
```

No módulo de OOP: usaremos `localStorage` para que os dados do To-Do List persistam entre sessões. O fluxo será: dados nas classes → `JSON.stringify` → `localStorage` → ao recarregar: `JSON.parse` → recriar instâncias com o constructor.

Slide 45 de 45

Resumo — O que Levar para o Módulo de OOP

Conceitos fundamentais que serão usados diretamente

Aqui está um mapa de como cada conceito desta base se conecta ao módulo de OOP:

Conceito da Base	Onde aparece em OOP
const/let	Declarar instâncias e variáveis locais nos métodos
Tipos e typeof	Validações nos setters (verificar se é string, number, etc.)
Referência vs valor	Entender por que retornar cópia [...array] nos getters
===	Comparações corretas em validações e buscas
Truthy/Falsy	Validações rápidas: if (!nome) throw Error(...)
if/else e guard clause	Validações em métodos e setters
Funções e return	Base para entender métodos de classe
Arrow functions	Callbacks em map, filter, forEach dentro dos métodos
Valores padrão	Parâmetros do constructor com defaults
Array methods	filter (deletar), find (buscar), map (transformar), reduce (somar)
Objetos literais	Base conceitual para entender classes
Template literals	Exibir informações formatadas nos métodos
Spread [...]	Retornar cópias seguras de arrays internos
try/catch e throw	Validações robustas nos setters
String methods	Validações: trim(), includes('@'), toUpperCase()
Math e Number	Cálculos: Math.PI, toFixed(2), Math.random()
Set	Armazenar letras chutadas no jogo da forca
Date	Atributos dataCriacao, dataValidade, comparações

JSON	Serializar para localStorage e APIs
DOM	Conectar classes à interface visual
Eventos	Capturar ações do usuário e chamar métodos das classes
localStorage	Persistir dados do To-Do List entre sessões

Se algum desses tópicos não ficou claro, **revise antes da Aula 1**. Cada um será usado diretamente no módulo. Com essa base sólida, OOP vai fazer total sentido!

Total de 45 slides